AIR UNIVERSITY, AEROSPACE AND AVIATION CAMPUS, KAMRA

DEPARTMENT OF COMPUTER SCIENCE

DIGITAL IMAGE PROCESSING LAB

BSCS – V

Name	MAIZA FATIMA
Registration ID	235205
Semester	6th
Date of Submission	3rd April , 2026 Friday
Submitted To	**Sir Ghulam**
LAB TASK:06	

OBJECTIVE:

The objective of this project is to design a complete system for image enhancement using all the techniques from previous labs such as image acquisition, sampling/quantization, geometric transformations, intensity transformations, and histogram equalization to enhance a low contrast image.

Task No:01

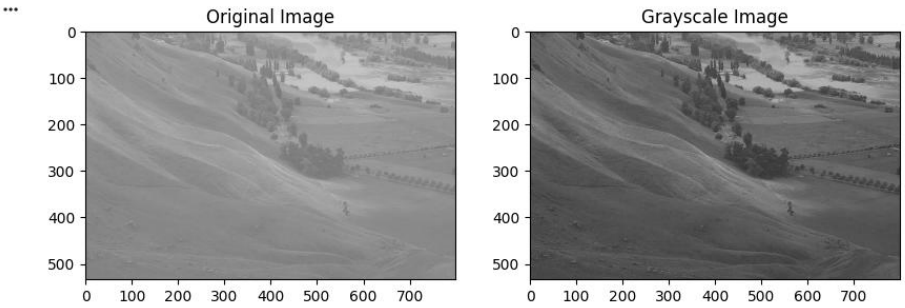
```
from google.colab import files
uploaded = files.upload()
```

```
Choose files hawkes_bay_in.jpg
hawkes_bay_in.jpg(image/jpeg) - 54649 bytes, last modified: 27/03/2026 - 100% done
Saving hawkes_bay_in.jpg to hawkes_bay_in.jpg
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow
```

Task 6.1

```
img = cv2.imread('hawkes_bay_in.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title('Original Image')

plt.subplot(1,2,2)
plt.imshow(gray, cmap='gray')
plt.title('Grayscale Image')
plt.show()

print('Image Matrix (partial):')
print(gray[0:5, 0:5])

print('Resolution:', gray.shape)
print('Data Type:', gray.dtype)
```




```
Image Matrix (partial):
[[125 149 160 155 134]
 [181 150 130 123 148]
 [155 141 150 132 142]
 [135 133 152 140 139]
 [146 144 145 137 123]]
Resolution: (534, 800)
Data Type: uint8
```

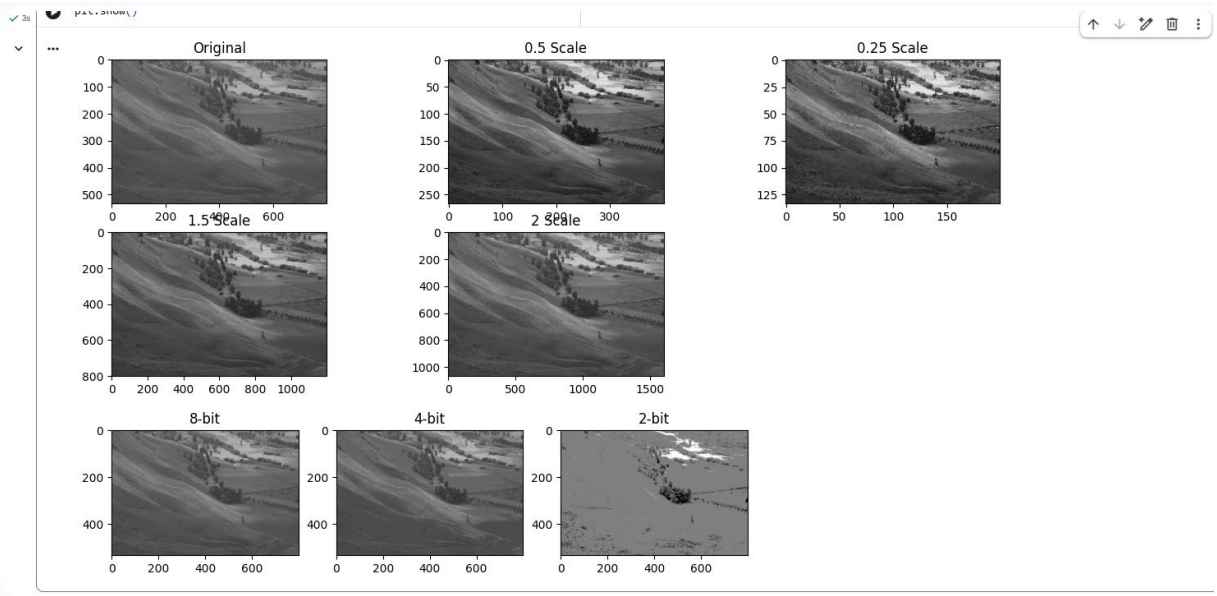
In this task, the image ‘hawkes_bay’ was loaded using the ‘cv2.imread()’ function. The image was converted into grayscale using ‘cv2.cvtColor()’. The original image and the grayscale image were displayed side by side. The image matrix, resolution, and data type were also printed in the console.

Task No: 02

Task:02

[6]
3s

```
plt.figure(figsize=(15,5))

img_05 = cv2.resize(gray, (0,0), fx=0.5, fy=0.5)
img_025 = cv2.resize(gray, (0,0), fx=0.25, fy=0.25)
img_15 = cv2.resize(gray, (0,0), fx=1.5, fy=1.5)
img_2 = cv2.resize(gray, (0,0), fx=2, fy=2)

plt.subplot(2,3,1); plt.imshow(gray, cmap='gray');    plt.title('Original')
plt.subplot(2,3,2); plt.imshow(img_05, cmap='gray');  plt.title('0.5 Scale')
plt.subplot(2,3,3); plt.imshow(img_025, cmap='gray'); plt.title('0.25 Scale')
plt.subplot(2,3,4); plt.imshow(img_15, cmap='gray');  plt.title('1.5 Scale')
plt.subplot(2,3,5); plt.imshow(img_2, cmap='gray');   plt.title('2 Scale')
plt.show()

img_8bit = gray
img_4bit = (gray & 0xF0)
img_2bit = (gray & 0xC0)

plt.figure(figsize=(10,4))
plt.subplot(1,3,1); plt.imshow(img_8bit, cmap='gray'); plt.title('8-bit')
plt.subplot(1,3,2); plt.imshow(img_4bit, cmap='gray'); plt.title('4-bit')
plt.subplot(1,3,3); plt.imshow(img_2bit, cmap='gray'); plt.title('2-bit')
plt.show()
```



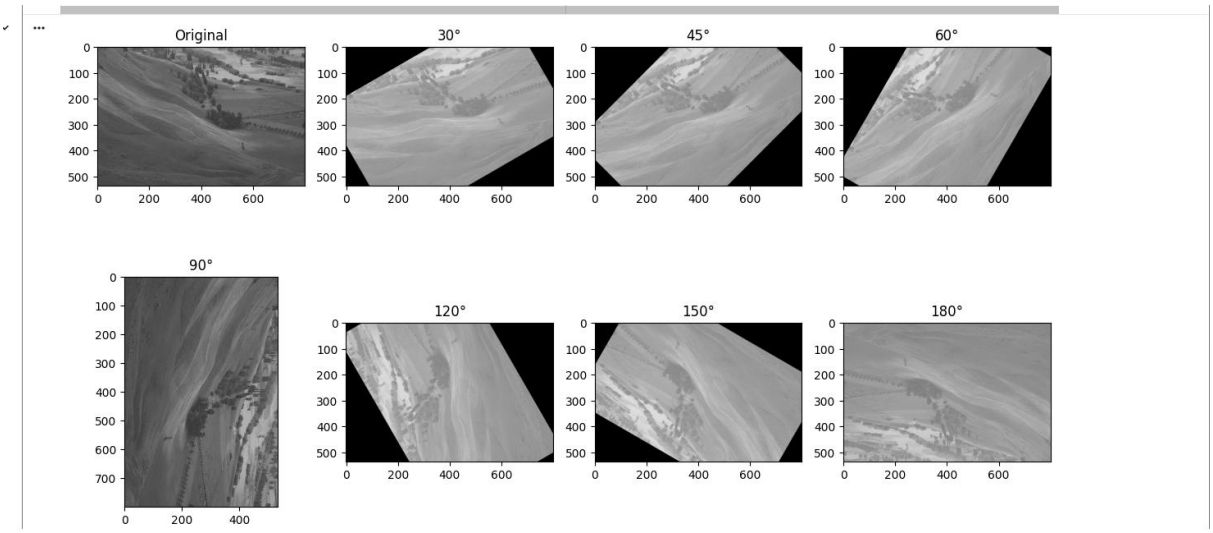
In this task, I have resized the image at different scales like 0.5, 0.25, 1.5, and 2 using the cv2.resize() function. In addition, I have reduced the bit depth to 8-bit, 4-bit, and 2-bit. I noticed that when the scale is reduced, the image becomes smaller. Moreover, when the bit depth is reduced, the quality of the image is reduced significantly.

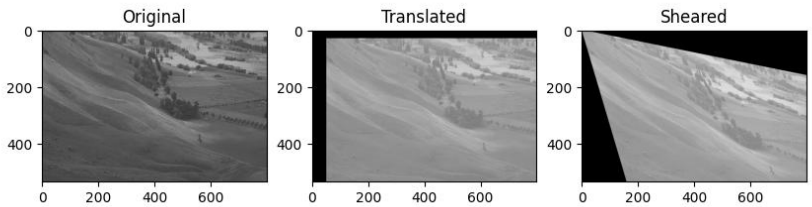
Task No: 03

Task 03

```
from scipy.ndimage import affine_transform

# Rotations
angles = [30, 45, 60, 90, 120, 150, 180]

plt.figure(figsize=(15,8))
plt.subplot(2,4,1)
plt.imshow(gray, cmap='gray')
plt.title('Original')

for i, angle in enumerate(angles):
    rotated = cv2.rotate(gray, cv2.ROTATE_90_CLOCKWISE) if angle==90 else cv2.warpAffine(gray, cv2.getRotationMatrix2D((gray.shape[1]//2, gray.shape[0]//2), angle, 1), (gray.shape[1
    plt.subplot(2,4,i+2)
    plt.imshow(rotated, cmap='gray')
    plt.title(f'{angle}°')

plt.show()

# Translation
M = np.float32([[1,0,50],[0,1,30]])
translated = cv2.warpAffine(gray, M, (gray.shape[1], gray.shape[0]))

# Shearing
M_shear = np.float32([[1, 0.3, 0],[0.2, 1, 0]])
sheared = cv2.warpAffine(gray, M_shear, (gray.shape[1], gray.shape[0]))

plt.figure(figsize=(10,4))
plt.subplot(1,3,1); plt.imshow(gray, cmap='gray');       plt.title('Original')
plt.subplot(1,3,2); plt.imshow(translated, cmap='gray'); plt.title('Translated')
plt.subplot(1,3,3); plt.imshow(sheared, cmap='gray');    plt.title('Sheared')
plt.show()
```

2:53 PM Python 3



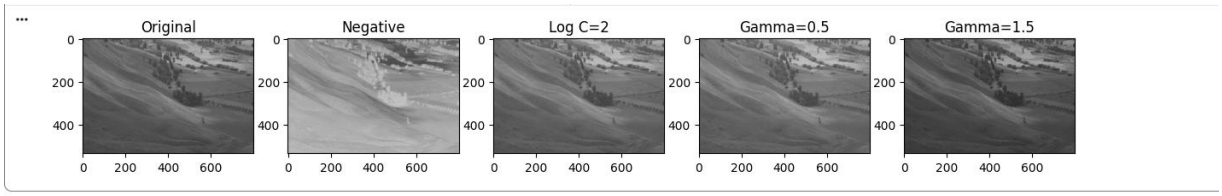


In this task, rotation was applied at seven different angles: 30, 45, 60, 90, 120, 150, and 180 degrees using the cv2.warpAffine() function. Translation and shearing were also applied to the image. It was noted that rotation changed the orientation of the image, translation changed the location of the image, and shearing changed the geometric orientation of the image.

Task No: 04

Task:04

```python
img_double = gray / 255.0

# Negative
negative = 255 - gray

# Log Transformation
c = 2
log_img = c * np.log1p(img_double)
log_img = log_img / log_img.max()

# Gamma Transformations
gamma1 = 0.5
gamma_bright = np.power(img_double, gamma1)

gamma2 = 1.5
gamma_dark = np.power(img_double, gamma2)

plt.figure(figsize=(15,4))
plt.subplot(1,5,1); plt.imshow(gray, cmap='gray');          plt.title('Original')
plt.subplot(1,5,2); plt.imshow(negative, cmap='gray');      plt.title('Negative')
plt.subplot(1,5,3); plt.imshow(log_img, cmap='gray');       plt.title('Log C=2')
plt.subplot(1,5,4); plt.imshow(gamma_bright, cmap='gray'); plt.title('Gamma=0.5')
plt.subplot(1,5,5); plt.imshow(gamma_dark, cmap='gray');    plt.title('Gamma=1.5')
plt.show()
```

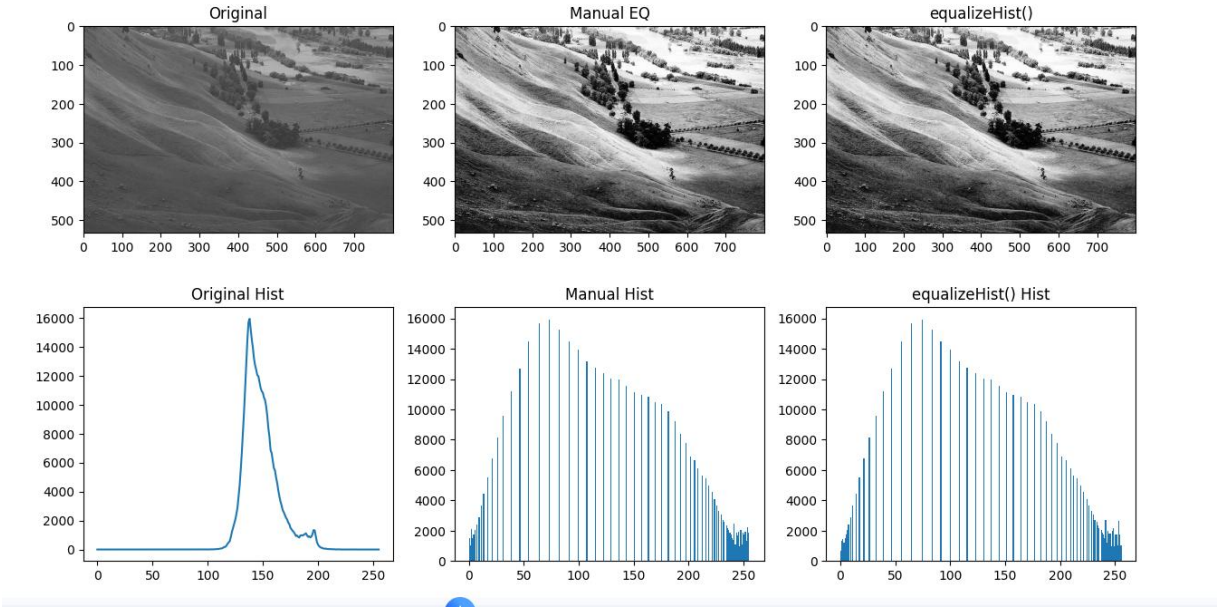


In this task, I used a negative transformation, log transformation with C = 2, and gamma transformation with values 0.5 and 1.5 on the grayscale image. I noticed that when I used a negative transformation, the pixel values were inverted. Log transformation made the dark regions brighter, while gamma 0.5 made the image brighter, and gamma 1.5 made the image darker.

Task No: 05

TASK:05

```python
hist_orig = cv2.calcHist([gray], [0], None, [256], [0,256])

pdf = hist_orig / hist_orig.sum()
cdf = np.cumsum(pdf)
L = 256
new_values = np.uint8((L-1) * cdf)
eq_manual = new_values[gray]

eq_histeq = cv2.equalizeHist(gray)

plt.figure(figsize=(15,8))
plt.subplot(2,3,1); plt.imshow(gray, cmap='gray');      plt.title('Original')
plt.subplot(2,3,2); plt.imshow(eq_manual, cmap='gray'); plt.title('Manual EQ')
plt.subplot(2,3,3); plt.imshow(eq_histeq, cmap='gray'); plt.title('equalizeHist()')

plt.subplot(2,3,4); plt.plot(hist_orig);                          plt.title('Original Hist')
plt.subplot(2,3,5); plt.hist(eq_manual.ravel(), 256, [0,256]);    plt.title('Manual Hist')
plt.subplot(2,3,6); plt.hist(eq_histeq.ravel(), 256, [0,256]);    plt.title('equalizeHist() Hist')
plt.show()
```

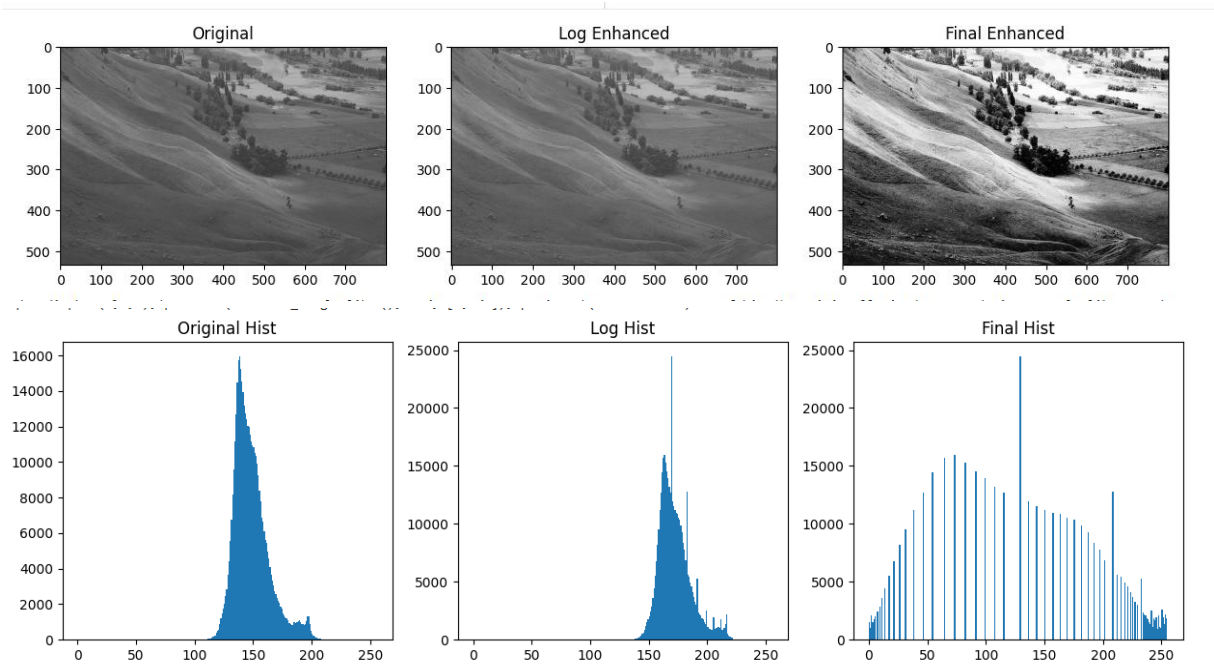


In this task, I plotted the histogram of the original image, equalized the histogram using manual methods with the PDF and CDF formulas, and used the OpenCV function equalizeHist(). I noticed that the original image histogram was only distributed over a narrow range, indicating a low contrast image. However, the histogram was distributed across the whole range after equalization.

Task No: 06

TASK:06

```
[10]
✓ 1s
img_double = gray / 255.0

c = 2
log_img = c * np.log1p(img_double)
log_img = log_img / log_img.max()
log_uint8 = np.uint8(log_img * 255)

hist = cv2.calcHist([log_uint8], [0], None, [256], [0,256])
pdf = hist / hist.sum()
cdf = np.cumsum(pdf)
L = 256
new_values = np.uint8((L-1) * cdf)
enhanced_image = new_values[log_uint8]

plt.figure(figsize=(15,5))
plt.subplot(1,3,1); plt.imshow(gray, cmap='gray');           plt.title('Original')
plt.subplot(1,3,2); plt.imshow(log_uint8, cmap='gray');      plt.title('Log Enhanced')
plt.subplot(1,3,3); plt.imshow(enhanced_image, cmap='gray'); plt.title('Final Enhanced')
plt.show()

plt.figure(figsize=(15,4))
plt.subplot(1,3,1); plt.hist(gray.ravel(), 256, [0,256]);           plt.title('Original Hist')
plt.subplot(1,3,2); plt.hist(log_uint8.ravel(), 256, [0,256]);      plt.title('Log Hist')
plt.subplot(1,3,3); plt.hist(enhanced_image.ravel(), 256, [0,256]); plt.title('Final Hist')
plt.show()
```



In this task, an entire image enhancement pipeline was created using a combination of log transformation and histogram equalization. Log transformation was applied first to enhance the dark areas of the image. After that, histogram equalization was applied to enhance the entire range of pixel intensities in the image. It was noted that the final enhanced image has better contrast compared to the original image.

Question/Answers:

Q1: Why does histogram equalization improve contrast?

Histogram equalization improves the contrast of the image because the intensity distribution is now across the full range from 0 to 255. In a low contrast image, the intensity distribution is only across a narrow range, which causes the image to appear dull. But after equalization, the intensity is distributed uniformly across the range, making the image appear clear.

Q2: How does gamma affect brightness?

Gamma transformation follows the formula s = c x r^γ. When gamma is less than 1, the output image becomes brighter because dark pixel values are expanded to higher intensity levels. When gamma is greater than 1, the output image becomes darker because bright pixel values are compressed to lower intensity levels. Therefore gamma directly controls the brightness of the image.

Q3: What is the impact of quantization on the quality of the image?

The quantization level decreases the number of bits used for the intensity value of the pixels. At 8-bit depth, the intensity level is 256. The resulting image is smooth and detailed. However, when the depth is reduced to 4-bit or 2-bit levels, the intensity level is reduced significantly. This results in the degradation of the quality of the image.

Q4: Which of the above transformations is reversible and why?

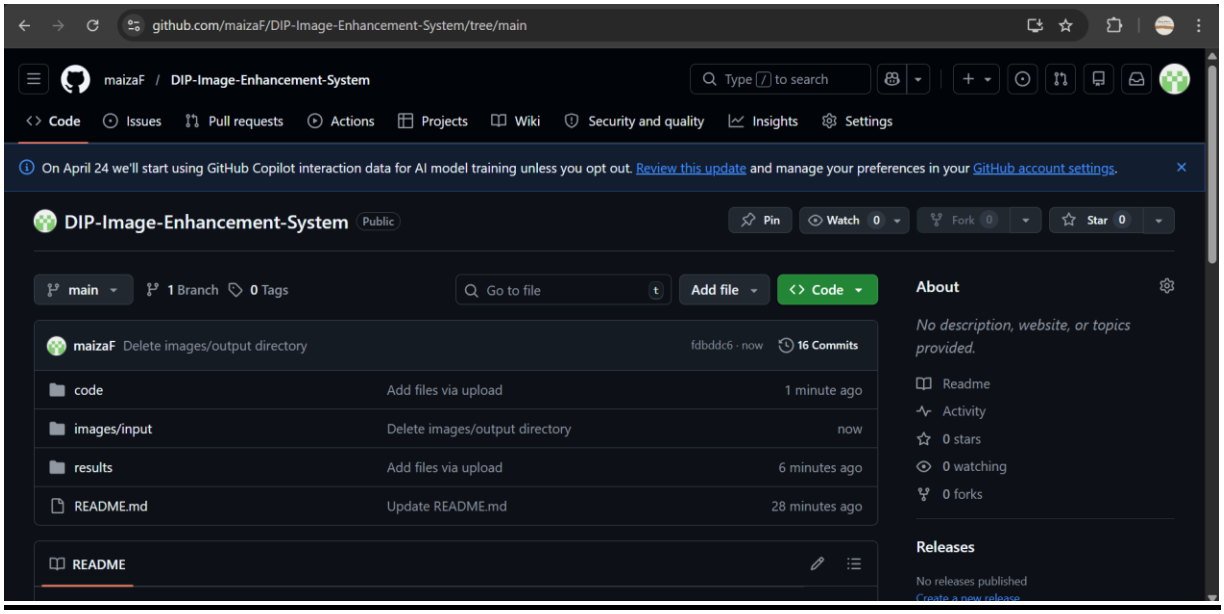
The rotation transformation is reversible because when the inverse rotation is applied with the same angle as the original rotation, the original image is retrieved without any information loss. For example, when the original image is rotated by 45 degrees, the resulting image is retrieved when the original image is rotated back by -45 degrees. Similarly, the translation is reversible with the same reason.

Q5: How do transformations affect spatial structure?

The geometric transformations include rotation, translation, and shearing. These transformations change the position of pixels in an image. However, the intensity of pixels remains unchanged. In other words, the shape of objects in an image changes, but the intensity of pixels remains the same. On the other hand, intensity transformations include negative transformation, log transformation, and gamma transformation. These transformations change the intensity of pixels but do not change the position of pixels.

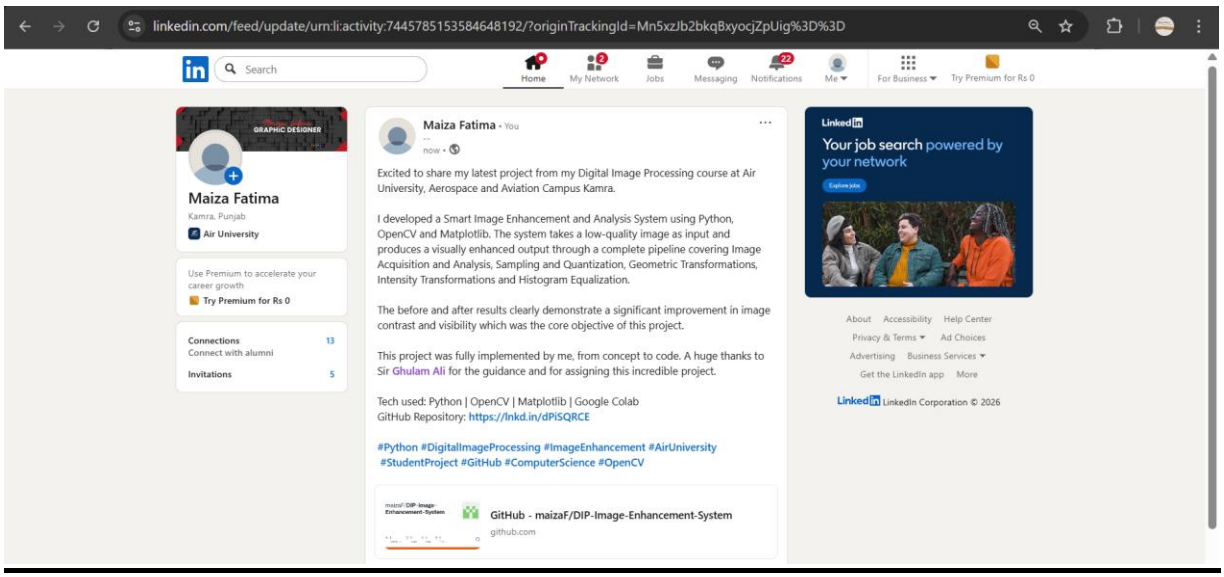
GitHub:

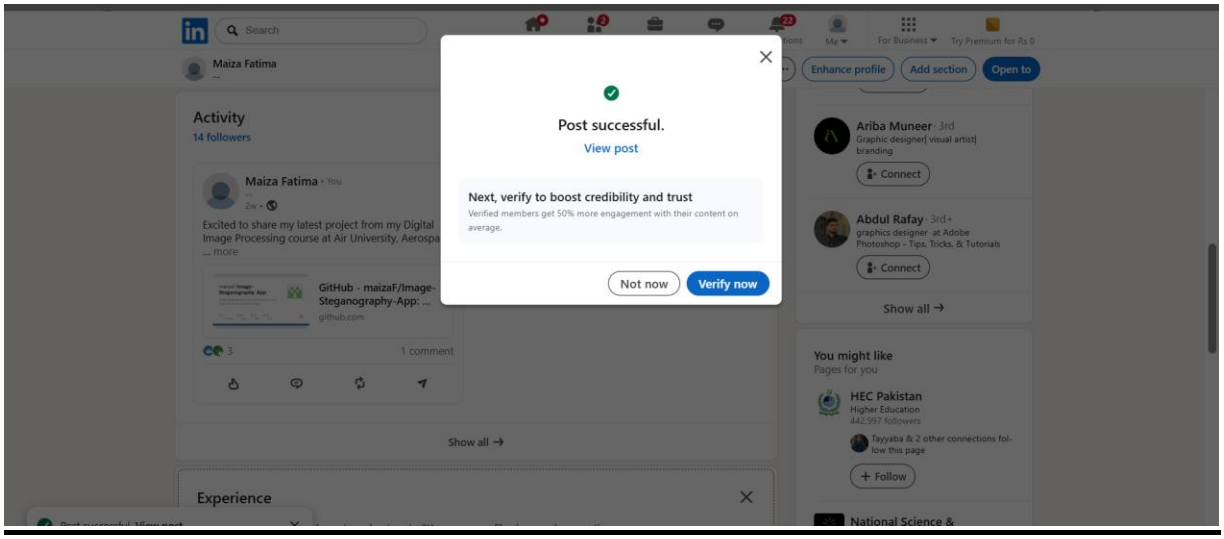
My Project is uploaded in github.
Link: https://github.com/maizaF/DIP-Image-Enhancement-System



LinkedIn:

Link: https://www.linkedin.com/in/maiza-fatima-5749a92ab/?skipRedirect=true





Conclusion:

In this project, I was able to successfully design and develop a complete Smart Image Enhancement and Analysis System using Python, OpenCV, and Matplotlib. The system accepts a low-quality image as input and produces a visually enhanced image as output. In the project, I used sampling and quantization to study the effect of resolution and bit depth, geometric transformations like rotation, translation, and shearing to study the effect of spatial changes, intensity transformations like negative, log, and gamma to study the effect of brightness, and histogram equalization to study the effect of intensity distribution. The enhanced image was significantly improved in contrast compared to the original image. In this project, I was able to understand how different image processing techniques can be combined to develop a complete and professional image enhancement system.